

DRYDOCK



AI Systems Architecture Review

Agentic pipeline generation and validation harness — client feed specs in, tested ingestion pipelines out, nothing sails without a signature

Release/ship date · 12 September 2026 · Roshan Rana

Executive summary

What it is

- A bounded LangGraph state machine that plans through MCP tools, generates a pipeline + Airflow DAG + field mapping from a client's feed specification, tests the result in a sandbox against six deterministic checks, repairs on failure, and stops at a real human gate before anything is published.

What it proves

- The loop is provably bounded, auditable and resumable across processes: every step is a LangGraph checkpoint in SQLite, and the human-approval interrupt cannot be skipped by any flag, config or provider.
- 6/6 corpus scenarios replay as declared; 5/5 adversarial pipelines are rejected; the gate runs offline in one command.

Why it is enterprise-ready

- Built through a gated, evidence-producing lifecycle: frozen LLD contracts, 12 task packs, code and security review, a documented three-layer sandbox hardened after an independent security review found real escapes.

Ask of the audience: agree the pilot scope (a recorded live-model run, the Docker sandbox as default, HMAC-signed approvals) and which provider (Ollama, vLLM, Bedrock, Anthropic) a first client's data perimeter should target.

The problem and the users

The onboarding problem

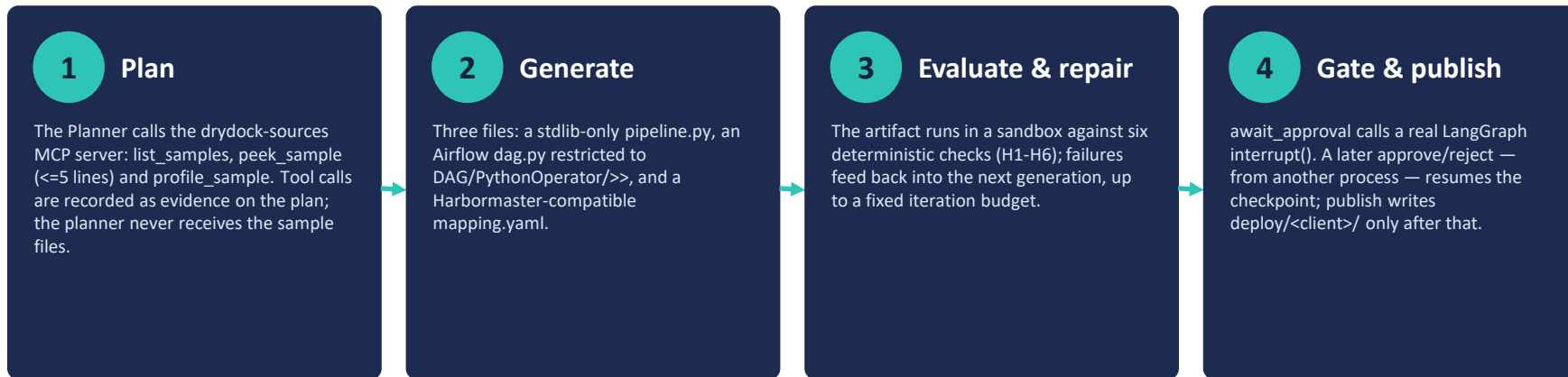
Every client feed arrives with a specification that is nearly right and a sample file that is nearly what the specification describes. Someone reads both, writes a parser, runs it, finds the trailer row counted as data, fixes it, and asks a colleague to sign off in a chat message. The work is repetitive, the fixes are undocumented, and the sign-off leaves no evidence.

The trust constraint

Language models can write the parser; the question is whether anything they write can be trusted into a scheduler. DRYDOCK's answer: the model's output is a proposal, a deterministic harness is the judge, and a human is the only thing that can publish.

Actor	Needs	Frequency
Forward-deployed engineer	A drafted, tested pipeline + DAG + mapping per client feed, with the repair history kept	Per client onboarded
Reviewer / approver	Iteration reports with evidence, an approve/reject gate that cannot be bypassed	On every run
Platform / ops	Deterministic gate offline; swappable model backend; no secrets in config	Deploy / operate
Automation / AI assistants	The same build/inspect/approve surface over CLI, dashboard and MCP	Ad hoc

Solution at a glance



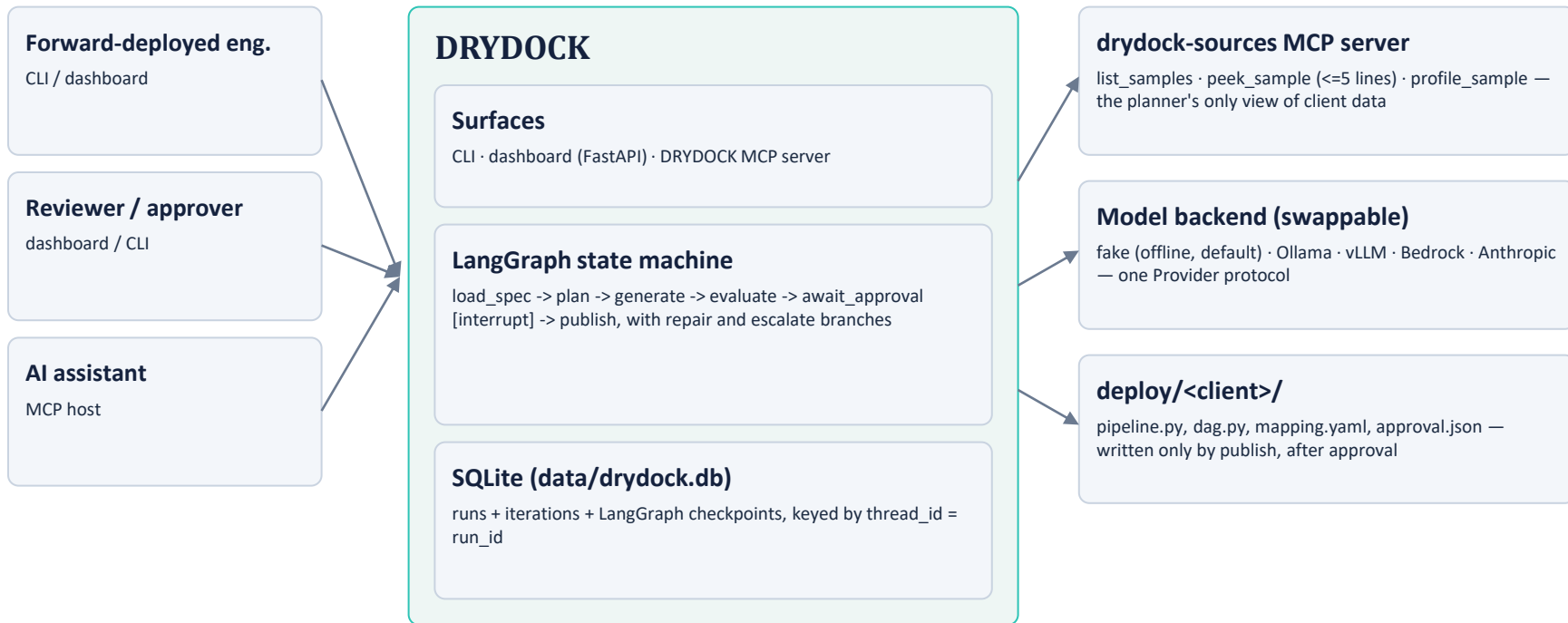
Surfaces

CLI (typer): `build`, `runs`, `show`, `approve`, `reject`, `replay`. Dashboard (FastAPI + vanilla HTML/JS): `runs`, `diffs`, `checkpoint history`. DRYDOCK MCP server: `build/inspect/approve` as 7 tools. All four read and write through one `graph.service`.

Deterministic by default, live by explicit act

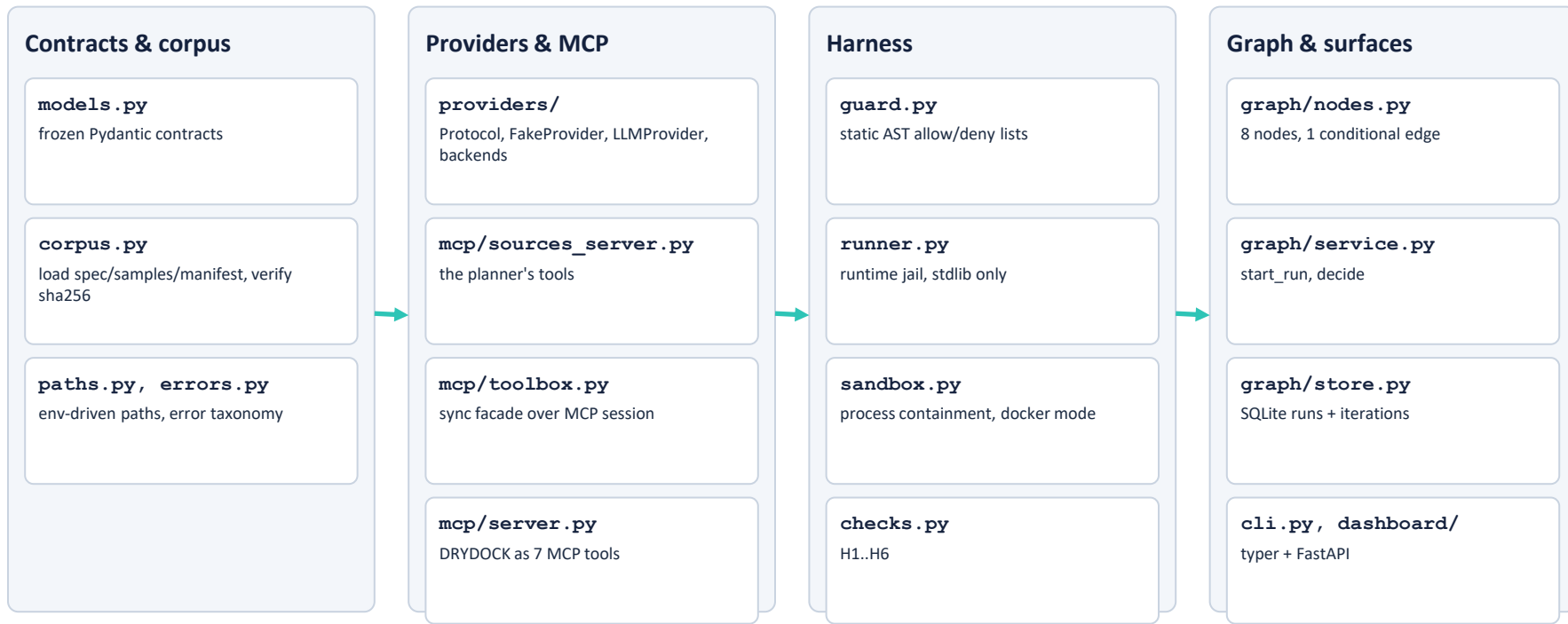
The fake provider drives the tests, the bench and CI: templates plus seeded fault injection, zero network calls. Ollama, vLLM, Bedrock and Anthropic are configuration files plus an environment variable, never a side effect of having a key present.

System context



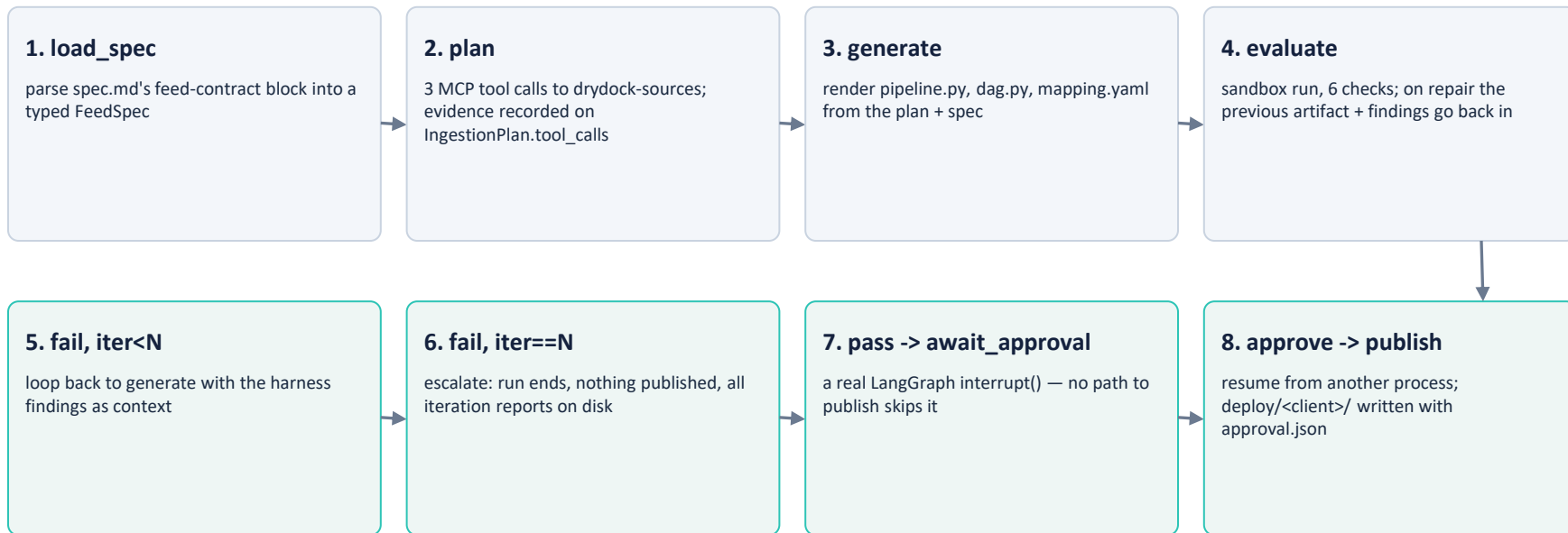
The corpus (corpus/<client>/spec.md, samples/, manifest.json) is the only client-owned input; everything the model sees is mediated by the sources server, never handed as whole files.

Component architecture



Generated code never imports into the orchestrator: the harness copies `pipeline.py` and `dag.py` into a scratch directory and runs them in a fresh interpreter. `models.py` and the LLD are frozen contracts task packs code against without talking to each other.

The critical flow: plan, generate, evaluate, gate



Every node appends one line to `runs/<run_id>/events.jsonl` (node, iteration, ms, outcome). The bench replays all 5 flows (happy, self-heal, escalate, reject, cross-process resume) with a fixed seed.

The core mechanism: agents propose, the harness decides, a human disposes

await_approval — the actual gate

```
def await_approval(self, state):  
    # ...mark AWAITING_APPROVAL, log event  
    answer = interrupt(  
        approval_payload(state)  
    )  
    decision, approver, note = (  
        parse_decision(answer, state)  
    )  
    store.update(  
        run_id, approved_by=approver,  
        decision_note=note,  
    )
```

publish is reachable only through approve. No flag, config or provider skips it.

Pass, iter 1

the deterministic harness found nothing wrong — straight to the gate

Fail, then pass

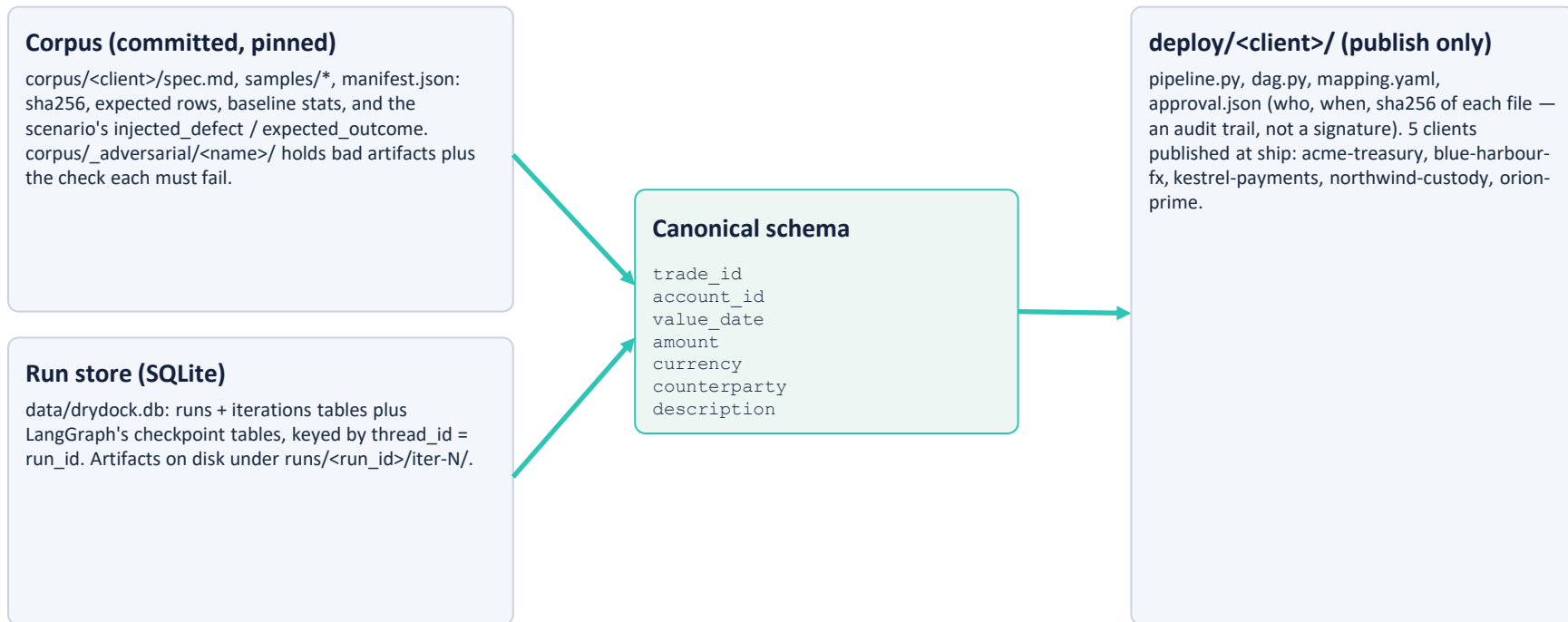
findings feed the next generation; repaired within the iteration budget

Fail x N, escalate

a spec that contradicts its sample cannot be fixed by better code — a person gets the contradiction, not a forced pass

The escalate scenario (meridian-legacy) is a wrong spec, not a broken generator: the pin is the client's asserted row count, so correct code fails three times and a person sees the contradiction. That is the behaviour a bank wants.

Data architecture: corpus, run store, deploy



The manifest is the oracle, not a reference implementation: the corpus author's parser builds it offline, the harness never calls it at run time, so a generator cannot pass by reproducing it.

Swappable model backends, one contract

Provider	Backend	Runs where	Credential
fake (default)	templates + seeded fault injection	in process	none
ollama	openai_compat over httpx	your machine	OPENAI_API_KEY (any value)
vllm	openai_compat over httpx	your GPU host	OPENAI_API_KEY (server's key)
bedrock	boto3 converse API	your AWS account	AWS credential chain, AWS_REGION
anthropic	anthropic SDK	Anthropic's API	ANTHROPIC_API_KEY

Data perimeter (what leaves the process)

Feed contract (columns, formats, quirks); column profiles (row count, amount total, null rates); sample name/size/sha256; at most 5 peeked raw lines per sample; on repair only, the previous artifact plus harness findings. Never sent: whole sample files, anything under deploy/, the SQLite store, credentials.

The provider changes who writes the code, nothing else

It changes nothing about the harness, the sandbox, the six checks, the iteration budget or the approval gate. `configs/providers/*.yaml` name a backend, a model and the environment variable holding a credential — never the credential itself.

Design choices and why

Decision	Alternatives considered	Why this one
LangGraph + SQLite checkpointer (ADR-002)	Hand-rolled state machine	Interrupts, checkpoint history and cross-process resume are requirements, not conveniences; a hand-rolled machine (as a sibling repo found) does not give all three
Python mcp 2.x, in-memory transport for tests (ADR-003)	Always-stdio MCP, a mocked tool layer	A real MCP session runs in the offline test suite with no spawned process; stdio only when the sources server runs standalone
Airflow validated via a stub package (ADR-004)	Install real Airflow; AST-only DAG check	Installing Airflow costs minutes and hundreds of MB and is irrelevant to the skill being shown; a stub records the declared structure
Raw backend adapters, not LangChain chat models (ADR-005)	LangChain chat model wrappers	Three adapters under 80 lines each behind one ChatBackend protocol make the local-vs-cloud story explicit and keep the base install small
Fault injection lives in the corpus manifest (ADR-006)	Fault injection in the generator itself	Each scenario declares injected_defect / expected_outcome; the unfixable scenario is a spec that contradicts its sample, keeping the demo honest
Three-layer sandbox (ADR-008)	Static AST guard alone	A security review showed the guard alone was not a boundary (an attribute hop reached os.system); defence in depth: guard, runtime jail, process containment

Major features

CLI (typer)

build, runs, show, approve --approver, reject --note, replay --step N, serve, mcp, bench

Review dashboard

FastAPI JSON API + dependency-free HTML trace viewer: iteration timeline, findings, code tabs with diff, checkpoint list, approve/reject form

DRYDOCK MCP server

7 tools over stdio; approve/reject enforce the gate before RunService is called; a run started from an MCP client shows up in the CLI and dashboard

Sources MCP server

list_samples, peek_sample (<=5 lines, path traversal rejected), profile_sample — the planner's only view of client data

Bench + results card

6 corpus scenarios + 5 adversarial artifacts replayed with seed 42; metrics/headline.json drives the README table and metrics.svg; CI fails on drift

Three-layer sandbox

static AST guard, runtime jail (jailed open, capability-stripped os), process containment (tree kill, rlimits, Windows Job Object, optional Docker)

The review dashboard: a self-heal run end to end

The screenshot displays the DRYDOCK dashboard for a pipeline named 'blue-harbour-fx'. The dashboard is divided into several sections:

- Header:** 'DRYDOCK' logo, 'agentic pipeline generation - human gate before anything sails', 'auto-refresh 5 s', and a 'Refresh' button.
- Runs List:** Shows two runs. The first run, 'blue-harbour-fx-20260907220424...', is 'approved'. The second run, 'acme-treasury-202609072...', is 'awaiting approval'.
- Run Details:** For the 'blue-harbour-fx' run, it shows 'client blue-harbour-fx', 'provider fake', 'iterations 2 / 3', and 'updated 2026-09-07 22:04:45Z'.
- Iteration Timeline:** Shows two iterations. Iteration 1 failed, and Iteration 2 passed. Each iteration shows a timeline of checks (H1, H2, H3, H4, H5, H6) and their results (rows, ms, fake).
- Findings:** Iteration 2 passed all six checks with no findings.
- Code:** Iteration 2 code is displayed, showing the ingestion pipeline for blue-harbour-fx / fx_settlements (csv).

Iteration 1 Details:

Check	Result	Rows	ms	fake
H1	passed	15	170	false
H2	failed			
H3	failed			
H4	failed			
H5	passed			
H6	passed			

Iteration 2 Details:

Check	Result	Rows	ms	fake
H1	passed	14	156	false
H2	passed			
H3	passed			
H4	passed			
H5	passed			
H6	passed			

Findings: iteration 2
All six checks passed with no findings.

Code: iteration 2
pipeline.py dag.py mapping.yaml

```
"""Ingestion pipeline for blue-harbour-fx / fx_settlements (csv).

Generated by DRYDOCK from the client feed contract. Standard library only: no third-party
imports, no network access, no file writes. dag.py wires extract -> transform -> load; the
DRYDOCK harness imports this module and calls extract() and transform() directly.

Canonical output, one dict per row with keys in this order:
    trade_id, account_id, value_date (YYYY-MM-DD), amount (signed, two decimals, no
    thousands separator), currency (ISO-4217 upper-case), counterparty, description
"""

import csv
from dataclasses import dataclass
from datetime import datetime
from decimal import ROUND_HALF_UP, Decimal

# ----- #
# Feed parameters (from the feed contract)
# ----- #

SOURCE_FORMAT = "csv"
DRYDOCK_HARNESS = ...
```

blue-harbour-fx iteration 1 parsed the trailer row as data and failed H2, H3 and H4; iteration 2 passed all six checks and the run is approved with 2/3 iterations spent.

Findings are evidence, not a verdict

DRYDOCK agentic pipeline generation · human gate before anything sails

2 total

blue-harbour-fx-20260907220424... **approved**

blue-harbour-fx fake iter 2/3

acme-treasury-202609072... **awaiting approval**

acme-treasury fake iter 1/3

blue-harbour-fx-20260907220424-61bd8e **approved** client **blue-harbour-fx** provider **fake** iterations **2 / 3** updated 2026-09-07 22:04:52

ITERATION TIMELINE · H1 runtime · H2 schema · H3 completeness · H4 drift · H5 latency · H6 DAG contract

Iteration 1 **failed**

H1 H2 H3 H4 H5 H6

15 rows 170 ms fake

Iteration 2 **passed**

H1 H2 H3 H4 H5 H6

14 rows 156 ms fake

FINDINGS iteration 1

CHECK	SEVERITY	MESSAGE	EVIDENCE
H2 schema	error	fx_settlements_20260904.psv: 1 row(s) violate the canonical schema	("sample": "fx_settlements_20260904.psv", "violations": 1, "offe...
H3 completeness	error	fx_settlements_20260904.psv: expected 14 rows, got 15	("sample": "fx_settlements_20260904.psv", "expected": 14, "actua...
H4 drift	error	fx_settlements_20260904.psv: null rate of value_date is 0.0667, baseline 0.0000	("sample": "fx_settlements_20260904.psv", "column": "value_date...
H4 drift	error	fx_settlements_20260904.psv: null rate of currency is 0.0667, baseline 0.0000	("sample": "fx_settlements_20260904.psv", "column": "currency"...
H4 drift	error	fx_settlements_20260904.psv: null rate of counterparty is 0.0667, baseline 0.0000	("sample": "fx_settlements_20260904.psv", "column": "counterpar...
H4 drift	error	fx_settlements_20260904.psv: null rate of description is 0.0667, baseline 0.0000	("sample": "fx_settlements_20260904.psv", "column": "descriptio...
H4 drift	error	fx_settlements_20260904.psv: 15 distinct trade_id, baseline 14	("sample": "fx_settlements_20260904.psv", "column": "trade_id"...
H4 drift	error	fx_settlements_20260904.psv: 6 distinct account_id, baseline 5	("sample": "fx_settlements_20260904.psv", "column": "account_id...
H4 drift	error	fx_settlements_20260904.psv: 5 distinct value_date, baseline 4	("sample": "fx_settlements_20260904.psv", "column": "value_date...
H4 drift	error	fx_settlements_20260904.psv: 15 distinct amount, baseline 14	("sample": "fx_settlements_20260904.psv", "column": "amount", "e...
H4 drift	error	fx_settlements_20260904.psv: 6 distinct currency, baseline 5	("sample": "fx_settlements_20260904.psv", "column": "currency"...
H4 drift	error	fx_settlements_20260904.psv: 9 distinct counterparty, baseline 8	("sample": "fx_settlements_20260904.psv", "column": "counterpar...
H4 drift	error	fx_settlements_20260904.psv: 15 distinct description, baseline 14	("sample": "fx_settlements_20260904.psv", "column": "descriptio...

What you are looking at

H2 schema and H3 completeness fail because the trailer row (a record-count footer) was parsed as a data row; H4 drift then shows 11 cascading findings — null rates and distinct counts thrown off by that one bad row.

Why this matters

This is exactly what feeds the repair: the generator's second attempt receives these findings, not a vague "it failed". The pinned baseline (14 rows) comes from a manifest the generator never sees.

The approval gate: checkpoints and the decision form

DRYDOCK

agentic pipeline generation - human gate before anything sails

auto-refresh 5 s Refresh

```
from dataclasses import dataclass
from datetime import datetime
from decimal import ROUND_HALF_UP, Decimal

# ----- #
# Feed parameters (from the feed contract) #
# ----- #

SOURCE_FORMAT = "csv"
DELIMITER = ","
ENCODING = "utf-8"
HEADER_ROWS = 1 # leading lines to skip; the last one names the columns
TRAILER_ROWS = 0 # trailing lines to skip (totals / record-count footer)
SKIP_BLANK_LINES = True

CANONICAL_COLUMNS = (
```

CHECKPOINTS 6 recorded

#-1	__start__	unknown	iter 0	1f1ab07a-7dc8	2026-09-07 22:01:22Z
#0	__start__	planning	iter 0	1f1ab07a-1427	2026-09-07 22:01:22Z
#1	load_spec	planning	iter 0	1f1ab07a-0c22	2026-09-07 22:01:22Z
#2	plan	generating	iter 0	1f1ab07a-8774	2026-09-07 22:01:22Z
#3	generate	evaluating	iter 1	1f1ab07a-640b	2026-09-07 22:01:22Z
#4	evaluate	evaluating	iter 1	1f1ab07a-1aec	2026-09-07 22:01:23Z

DECISION iteration 1 passed all checks — awaiting a human

Approver

your name

Note

optional note recorded with the decision

Approve and publish

Reject

acme-treasury's 6 recorded checkpoints — start, load_spec, plan, generate, evaluate — each a LangGraph checkpoint, replayable step by step; the decision form refuses to submit without a named approver.

The gate, run: one command, offline

```
$ uv run python scripts/check.py

≡ ruff check          All checks passed!
≡ ruff format        90 files already formatted
≡ mypy               Success: no issues found in 34 source files
≡ mypy (linux target) Success: no issues found in 34 source files
≡ pytest             503 passed, 2 skipped, 97.46% coverage (required ≥ 80%)

≡ bench: uv run drydock bench
bench: seed=42 provider=fake max_iterations=3
acme-treasury        expected=pass   observed=pass   iterations=1 checkpoints=8
blue-harbour-fx      expected=heal   observed=heal   iterations=2 checkpoints=10
kestrel-payments     expected=heal   observed=heal   iterations=2 checkpoints=10
meridian-legacy      expected=escalate observed=escalate iterations=3 checkpoints=11
northwind-custody    expected=heal   observed=heal   iterations=2 checkpoints=10
orion-prime          expected=heal   observed=heal   iterations=2 checkpoints=10
dag_missing_dependency rejected failed=H6_dag_contract
drops_last_row        rejected failed=H3_completeness, H4_drift
sleeps_past_budget    rejected failed=H5_latency
slow_network_import   rejected failed=H1_runtime
wrong_amount_format   rejected failed=H2_schema, H4_drift
wrote metrics/headline.json: scenarios 6 / 6, adversarial 5 / 5, checkpoints 59

≡ bench drift        (git diff --exit-code) clean
≡ card drift         metrics card is current

all checks passed
```

uv run python scripts/check.py, captured during this review: ruff, mypy strict (host and Linux target), pytest with coverage, the bench replay of all 6 scenarios and 5 adversarial pipelines, and both drift checks.

Security and compliance controls

Layer	Threat	Control	Evidence
1. Static AST guard	Import of a dangerous module; an attribute hop to a real capability (os.path -> os.system)	Import allowlist; attribute/name denylists; forbidden-call list; runs before anything executes	test_guard_blocks_payload_before_execution
2. Runtime jail	Reads outside the scratch workdir leak host files into harness evidence, then into a repair prompt	open/io.open confined to reads inside the workdir; capability-stripped os; poisoned dangerous modules; import finder	test_*_outside_workdir_is_blocked
3. Process containment	A hung or forked child survives a timeout; unbounded memory	Process-group/session kill on timeout; POSIX rlimits; Windows Job Object (512 MiB); Docker: read-only mount, --cap-drop ALL, --network none	test_sleeping_child_tree_is_killed_on_timeout

What the security review found (2026-09-07)

3 critical, 4 high, 2 medium findings against drydock.harness.evaluate, all closed before ship (T-012). pip-audit over the locked dependency set: 0 known vulnerabilities.

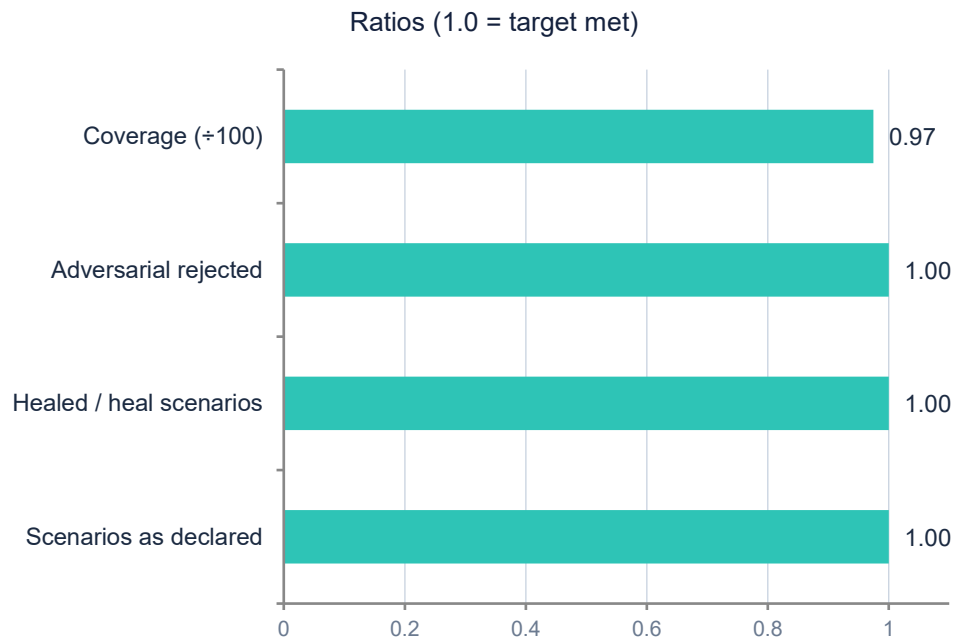
Residual, stated plainly

approval.json hashes are an audit trail, not a signature (M1). The Windows Job Object memory cap is best-effort; the wall-clock timeout is the backstop. The guard is name-based; layers 2 and 3 exist because of that.

How it was built: a gated, evidence-producing lifecycle



Quality metrics from the gate



KPI	Value	How
Tests / coverage	505 / 97.46%	check.py, this review
Scenarios as declared	6 / 6	bench, seed 42
Healed	4 / 4	4 heal scenarios
Adversarial rejected	5 / 5	hand-written artifacts
Mean iterations	2.00	per run, budget 3
Checkpoints	59	across 6 bench runs
Sandbox escape tests	25	test_sandbox_escapes.py

First-pass rate is 1/6 by design: 4 of 6 corpus scenarios inject a defect on purpose so the self-heal path is exercised, and one (meridian-legacy) is built to escalate. Live-provider, Docker-sandbox and real-Airflow rows on the results card are pending — no recorded run exists yet.

Quirks and known limitations (stated, not hidden)

The bench measures the harness, not a model

The fake provider is template-driven with scenario-declared defects; it says nothing about any real model's code quality. Live-provider accuracy is pending until a recorded run exists.

Docker sandbox mode is untested by the bench

The bench always uses subprocess mode, so its numbers are byte-identical with or without a Docker daemon; the docker=... kind is covered by dedicated tests, not by the headline KPIs.

Airflow is a stub offline

Generated DAGs are validated against a package that records structure at import; whether they load in a real scheduler is checked only by the opt-in docker compose --profile airflow target, and that result is pending.

approval.json is an audit trail, not a signature (M1)

Its sha256 hashes record what was approved and by whom but are not tamper-evident; anyone who can write deploy/<client>/ can rewrite both the artifact and its recorded hash.

Windows memory cap is best-effort (H4)

The Job Object caps committed memory at 512 MiB; if the ctypes call fails, only the wall-clock timeout backstops a memory bomb, which then ends as a timeout rather than a MemoryError.

The corpus is synthetic

Six clients and five adversarial cases exercise every branch of the graph and every check; they are not a survey of real feed formats.

The canonical schema is fixed at 7 columns

trade_id, account_id, value_date, amount, currency, counterparty, description. Real clients would extend it; that is unbuilt.

No multi-tenant auth

The dashboard and the MCP server are local, single-operator tools; SSO or per-tenant isolation is not built.

Cost, performance and operability

0 tokens

the fake provider drives the bench and CI;
a live provider records tokens per call but
none has been run yet

2.00

mean generate/evaluate iterations per
run against a budget of 3

30 s / 5 s

sandbox timeout per sample / H5 latency
budget per sample, both configurable

59

LangGraph checkpoints across 6 bench
runs — every step persisted and
replayable

Operability

- * runbook.md: start/stop, common failures
- * replay <run_id> walks every checkpoint; --step N dumps state at that step
- * events.jsonl: one line per node (node, iteration, ms, outcome) + one llm_usage line per real-model call
- * Rollback is deleting deploy/<client>; the run and its decision remain in SQLite and runs/

What a live run would add to this page

Prompt/completion tokens and latency per model call, written to events.jsonl by LLMProvider's on_usage callback. No aggregated cost figure exists in the repository today because no live run has been recorded; when one is, the sum over events.jsonl is the figure to report.

Roadmap

Now

Recorded

- 6/6 scenarios, 5/5 adversarial, 505 tests, 97.46% coverage, all offline with the fake provider

Next

A live-model run

- Record one build with Ollama or vLLM and publish its accuracy/cost/latency as an observed figure, replacing the pending row

Next

Docker sandbox as a bench input

- Run the bench under sandbox=docker so the results card reflects the OS-level isolation boundary, not just subprocess mode

Backlog

Hardening

- HMAC-signed approvals (approval.json is presently an audit trail, not a signature); real-Airflow bench row; add DRYDOCK to the profile README

Per STATE.md's backlog: none of the next steps are load-bearing for the current, offline gate — they add recorded evidence, not new architecture.

Appendix A — Decision log (abridged)

ID	Decision	Consequence
ADR-000	Design gates self-approved by the orchestrating agent, under the owner's autonomy instruction, for an interview-ready repo fast	Every gate artifact still exists and is reviewable after the fact
ADR-001	Name: DRYDOCK — a vessel is tested and inspected before the harbour	Sets the metaphor: Harbormaster controls what enters the harbour, DRYDOCK decides what is seaworthy
ADR-002	LangGraph with a SQLite checkpointer	Interrupts, checkpoint history and cross-process resume as first-class, zero-infra features
ADR-003	Python mcp 2.x, in-memory transport for tests	Planner tool calls exercised over a real MCP session with no spawned process in CI
ADR-006	Fault injection lives in the corpus manifest, not the generator	Keeps the self-heal and escalate demos honest and published, not hard-coded into the model
ADR-007	Four Wave-1 integration reconciliations	Tool kwarg mismatch, escalate pin, a shifted adversarial slice, and a split adversarial case — all fixed at integration, not before
ADR-008	Security findings drove a three-layer sandbox	Static guard alone was shown not to be a boundary; guard + runtime jail + process containment closed it

Appendix B — Build statistics and lessons

Item	Value
Task packs	12 (T-000 .. T-012, including sandbox hardening)
Wave schedule	Wave 1 x4 parallel (corpus, harness, providers, sources MCP) -> Wave 2 x4 (graph, dashboard, MCP server, docs) -> Wave 3 x2 (bench, Airflow) -> Wave 4 (integration + ship)
Code review	approve, 0 critical, 0 high; 1 medium fixed (unbounded errors column), 1 low documented (unused SandboxTimeout)
Security review	3 critical, 4 high, 2 medium before T-012; all closed same day; pip-audit clean
Tests / coverage at ship	505 tests (98% per ship-report; 503 passed / 2 skipped / 97.46% observed in this review)
Generated code lines	1689 across the final pipeline.py, dag.py, mapping.yaml of the 6 bench runs

Lessons written into the decision log

Integration-only defects surfaced where independently built components met (a tool kwarg mismatch, an escalate pin, a shifted adversarial slice) — reconciled in the integration wave, not before. The security review's own payloads are committed as regression tests, so a beaten guard cannot regress silently.

Appendix C — Repository map and how to run

drydock/	models.py, paths.py, errors.py, corpus.py
harness/	sandbox runner, six checks, stub airflow package
providers/	Provider protocol, FakeProvider, LLMPProvider, backends
mcp/	sources server, toolbox, DRYDOCK's own MCP server
graph/	nodes, edges, checkpointer, run store, service API
dashboard/	FastAPI JSON API + dependency-free HTML viewer
cli.py, bench.py, events.py	
corpus/	six synthetic clients and five adversarial pipelines
configs/providers/	fake, ollama, vllm, bedrock, anthropic
metrics/	render.py, card.json, headline.json (bench-written)
scripts/check.py	the gate: ruff, mypy, pytest, bench, drift checks
docs/design/	requirements, HLD, LLD, execution plan, decision log
docs/tasks/	T-000 .. T-012 task packs
docs/ship-report.md, docs/security.md, docs/serving.md, docs/runbook.md	

Run it

```
uv sync
uv run drydock build acme-treasury \
  --provider fake
uv run drydock runs
uv run drydock approve <run_id> \
  --approver <name>

# dashboard
uv run drydock serve --port 8601

# the whole gate, identical to CI
uv run python scripts/check.py
```

Questions

DRYDOCK · roshanrana/drydock · 505 tests, 97.46% coverage, one offline gate · design docs and ship report in the repository